

AI Workflow 技术手册

版本: v1.1

最后更新: 2026-06-11

文档路径: docs/aiworkflow-tech-manual.md

参考格式: BladeX 大模型平台技术手册

目录

第01章 序言

- 平台介绍
- 核心特性
- 技术栈
- 平台生态

第02章 架构讲解

- 整体架构
- 后端分层架构
- Store 模式详解
- 前端 Monorepo 架构
- 数据库设计
- 配置与 Profile 体系
- 安全与认证架构
- 性能调优

第03章 快速开始

- 环境要求
- 本地开发模式（无数据库）
- 标准开发模式（PostgreSQL）
- 启动前端
- 访问系统
- 工程部署

第04章 认证与多租户指南

- [认证体系概览](#)
- [管理端登录与 Token](#)
- [租户上下文与隔离](#)
- [组织用户与角色](#)
- [开放 API 认证](#)
- [资产授权](#)
- [审计日志](#)

第05章 工作流引擎指南

- [概念介绍](#)
- [工作流定义模型 \(DAG\)](#)
- [节点类型详解](#)
- [执行引擎原理](#)
- [DAG 校验机制](#)
- [模板渲染引擎](#)
- [版本管理与归档恢复](#)
- [API 参考](#)

第06章 知识库指南

- [概念介绍](#)
- [文档处理流程](#)
- [文档分割策略](#)
- [向量嵌入 \(Embedding\)](#)
- [向量存储与搜索](#)
- [知识库与工作流联动](#)
- [API 参考](#)

第07章 智能体指南

- [智能体架构](#)
- [会话管理](#)
- [执行模式详解](#)
- [Chat 对话流程](#)
- [API 参考](#)

第08章 模型指南

- [模型体系介绍](#)
- [模型类型与用途](#)
- [接入 Ollama 私有部署](#)
- [API 参考](#)

第09章 智能编研与开放集成

- [编研模板与 workflow 绑定](#)
- [智能编研任务链路](#)
- [第三方应用与 Java SDK](#)
- [设计器嵌入 \(React/Vue/WC\)](#)

第10章 二次开发指南

- [Store 模式扩展](#)
- [新增自定义节点类型](#)
- [自定义 Embedding 客户端](#)
- [自定义文档分割器](#)
- [Feign 外部服务集成](#)

第11章 API 接口参考

- [通用约定](#)
- [接口总览](#)
- [开放 API 规范](#)

第01章 序言

平台介绍

AI Workflow Platform 是一个全栈、可扩展的 AI 工作流自动化平台。用户可以通过可视化的 DAG（有向无环图）设计器构建复杂的工作流，连接大语言模型（LLM），构建知识库并实现向量搜索，创建 AI 智能体（ChatBot）进行多轮对话，并实时追踪每一次工作流执行的状态和节点级别的运行日志。

平台采用前后端分离架构，后端基于 **Spring Boot 3.3 + MyBatis-Plus** 构建，前端基于 **React 18 + Ant Design 5** 构建，数据库支持 **PostgreSQL**（默认）和**达梦数据库**（国产化适配）。

核心特性

工作流引擎 - 可视化 DAG 工作流设计器，支持拖拽式节点编排 - 12 种内置节点类型：开始、结束、LLM、Prompt、知识检索、HTTP 工具、条件分支、问题分类、文本转换、内容模板、循环 - 基于索引的版本管理，支持草稿/发布/归档/恢复发布状态流转 - 完整的执行追踪：每次运行记录输入/输出，每个节点记录执行状态和耗时 - 模板渲染引擎：支持 `{{ variable.path }}` 语法，全节点通用

知识库系统 - 5 种文档分割策略：固定长度、段落、语义、符号、结构化表格 - 文件上传支持（PDF、Word、Excel、Markdown、TXT 等）基于 Apache Tika - 多类型数据集：手动录入、文件上传、文本数据集、表格数据集 - 向量嵌入：支持本地哈希嵌入和 Ollama 远程嵌入 - 检索模式：关键词检索、向量检索（余弦相似度）、混合检索 - 向量存储支持 Elasticsearch

智能体 (Bot) - 多轮对话管理，自动创建/延续会话 - 工作流模式：Bot 绑定工作流，对话触发工作流执行 - 直接 LLM 模式：连接模型提供商进行直接推理 - RAG 模式：结合知识库实现检索增强生成 - 系统提示词、开场白自定义

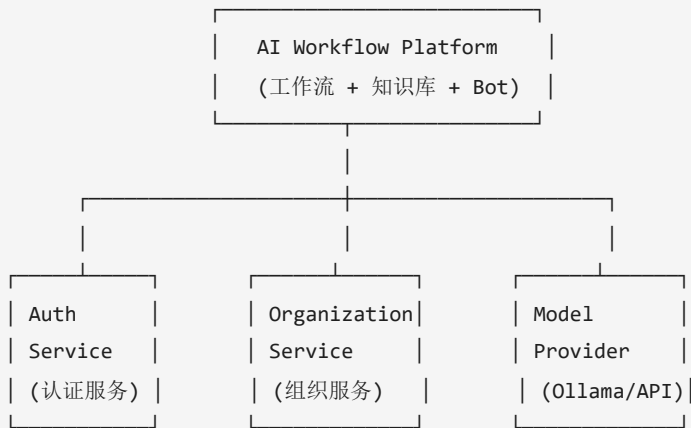
模型管理 - 统一的模型提供商管理界面 - 支持 Chat 模型和 Embedding 模型 - 支持 OpenAI 兼容 API 和 Ollama 私有部署 - 视觉支持、价格配置

平台特性 - 多租户与组织用户：租户、组织、用户、角色、登录会话 - 资产授权：按组织/部门/角色控制 Bot、知识库、工作流、模型可见性 - 开放 API： /api/open/** + AppCode/ApiKey 认证，供第三方系统集成 - 智能编研：编研模板 + 五步向导 + 工作流生成 Markdown/DOCX - 操作审计：关键管理类资源记录 created_by / updated_by - 多 Profile 体系：本地无数据库模式、PostgreSQL 标准模式、达梦国产化模式 - 可插拔持久化：同一套代码通过配置切换 InMemory/MyBatis 存储 - Nacos 服务发现（可选）：适配微服务架构 - Feign 声明式客户端：集成外部认证服务和组织架构服务 - 统一的 ApiResponse 响应格式 + 全局异常处理 - Swagger / OpenAPI 3.0 文档自动生成

技术栈

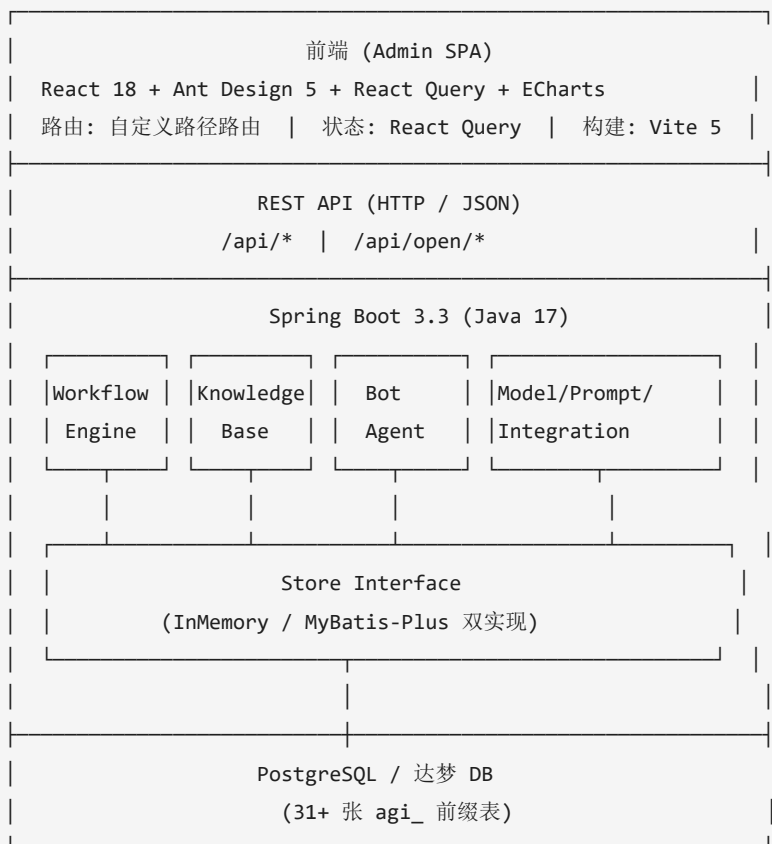
层级	技术选型	版本	说明
后端框架	Spring Boot	3.3.5	Java 17
API 文档	SpringDoc OpenAPI	2.6.0	Swagger UI
ORM	MyBatis-Plus	3.5.9	Spring Boot 3 适配
数据库迁移	Flyway	—	版本化 Schema 管理
数据库	PostgreSQL / 达梦	15+ / 8+	默认 PG，国产化达梦
服务发现	Nacos	2023.0.3.3	可选，默认关闭
远程调用	OpenFeign	—	声明式 HTTP 客户端
文档解析	Apache Tika	2.9.2	多格式文档提取
安全	Spring Security	—	CSRF 关闭，所有请求放行
前端框架	React	18.3	TypeScript 5.6
UI 组件库	Ant Design	5.21	中文 locale
服务端状态	@tanstack/react-query	5.59	请求缓存与状态管理
图表	ECharts	5.5	Dashboard 可视化
构建工具	Vite	5.4	前端构建
包管理	pnpm	—	Monorepo Workspaces
测试	Vitest / JUnit 5	—	前端 + 后端

平台生态



第02章 架构讲解

整体架构



核心设计理念:

- **前后端分离**: 前端通过 HTTP REST API 与后端通信, Vite Dev Proxy 代理 `/api` 到后端 8080 端口
- **接口/实现分离**: 每个业务模块的持久层通过 Store 接口抽象, 支持 InMemory 和 MyBatis 双实现切换
- **策略模式**: 工作流节点执行器通过 `WorkflowNodeExecutor` 接口实现, 每种节点类型一个策略类
- **注册表模式**: `WorkflowNodeExecutorRegistry` 自动发现并注册所有节点执行器

- **版本化 Schema 管理**：Flyway 管理数据库迁移，支持 PostgreSQL 和达梦双库

后端分层架构

每个业务模块采用统一的四层分层结构：

api/	← REST 控制器层 (Spring MVC)
├─ XxxController.java	@RestController, @RequestMapping
└─ DTO Records	请求/响应对象 (Java records)
domain/	← 领域模型层
├─ Xxx.java	领域实体 (Java records, 不可变)
└─ Enum	状态枚举
service/	← 业务逻辑层
├─ XxxService.java	核心业务逻辑
├─ XxxStore.java	持久层接口
├─ InMemoryXxxStore.java	ConcurrentHashMap 实现
└─ MybatisXxxStore.java	MyBatis-Plus 实现
persistence/	← 数据映射层
├─ XxxEntity.java	MyBatis-Plus 实体 (@TableName)
└─ XxxMapper.java	MyBatis-Plus Mapper 接口

项目包结构 (server/src/main/java/com/mw/ai/agi/)：

com.mw.ai.agi/	
├─ common/	# ApiResponse、GlobalExceptionHandler、OperatorContext
├─ config/	# OpenApiConfig、SecurityConfig、StoreConfig
├─ auth/	# 登录、租户、组织用户、Open API Filter
├─ asset/	# 资产授权
├─ bot/	# 智能体
├─ generation/	# 编研模板、编研任务
├─ integration/	# 第三方应用
├─ knowledge/	# 知识库
├─ model/	# 模型配置
├─ prompt/	# Prompt 模板
├─ system/	# 菜单、字典、审计日志
└─ workflow/	# 工作流引擎 (12 种 NodeExecutor)

Store 模式详解

Store 模式是本项目最核心的设计模式之一，允许在不修改业务代码的情况下切换持久化实现。

设计动机： - 本地开发/测试时不依赖数据库，使用内存存储快速启动 - 生产环境使用 PostgreSQL，国产化环境使用达梦数据库 - 同一套业务代码，零修改切换

实现原理：

StoreConfig.java 作为中心配置类，使用 Spring 的 ObjectProvider 检测 MyBatis Mapper Bean 是否可用：

```
@Bean
public WorkflowStore workflowStore(
    ObjectProvider<WorkflowMapper> workflowMapperProvider,
    ObjectProvider<WorkflowVersionMapper> versionMapperProvider,
    ObjectMapper objectMapper
) {
    WorkflowMapper workflowMapper = workflowMapperProvider.getIfAvailable();
    WorkflowVersionMapper versionMapper = versionMapperProvider.getIfAvailable();
    return workflowMapper == null || versionMapper == null
        ? new InMemoryWorkflowStore() // 无数据库时
        : new MybatisWorkflowStore( // 有数据库时
            workflowMapper, versionMapper, new JsonSupport(objectMapper));
}
```

决策流程：



涉及模块（共 7 组）：

Store 接口	InMemory 实现	MyBatis 实现	所属模块
WorkflowStore	InMemoryWorkflowStore	MybatisWorkflowStore	workflow
WorkflowExecutionStore	InMemoryWorkflowExecutionStore	MybatisWorkflowExecutionStore	workflow
KnowledgeStore	InMemoryKnowledgeStore	MybatisKnowledgeStore	knowledge
VectorStoreConfigStore	InMemoryVectorStoreConfigStore	MybatisVectorStoreConfigStore	knowledge
ModelProviderStore	InMemoryModelProviderStore	MybatisModelProviderStore	model
PromptTemplateStore	InMemoryPromptTemplateStore	MybatisPromptTemplateStore	prompt
BotStore	InMemoryBotStore	MybatisBotStore	bot

前端 Monorepo 架构

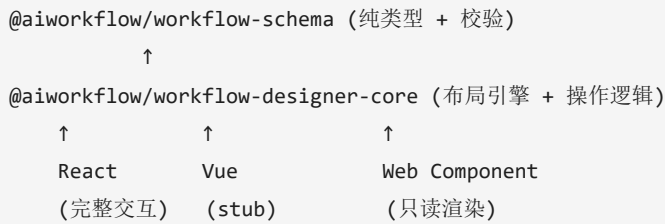
前端采用 pnpm workspaces Monorepo 架构，包含 1 个应用 + 5 个共享包：


```

web/
├─ apps/admin/                                # 管理后台 SPA
│   └─ src/
│       ├─ main.tsx                            # React 入口
│       ├─ App.tsx                            # 根组件: QueryClientProvider + 路由解析
│       ├─ routes.tsx                         # 路径路由表 (12 个已实现页面)
│       ├─ navigation.ts                      # pushState + popstate 导航
│       ├─ api/                               # API 模块 (fetch 封装)
│       │   ├─ workflows.ts                   # 工作流 API
│       │   ├─ bots.ts                       # 智能体 API
│       │   ├─ models.ts                     # 模型 API
│       │   ├─ knowledge.ts                  # 知识库 API
│       │   └─ prompts.ts                    # 提示词 API
│       ├─ layout/                            # 布局组件
│       │   ├─ AdminShell.tsx                # 侧边栏 + 主内容区
│       │   ├─ PageHeader.tsx               # 页头
│       │   └─ menu.ts                       # 菜单定义
│       └─ pages/                             # 页面组件
│           ├─ DashboardPage.tsx
│           ├─ workflows/
│           │   ├─ WorkflowCardsPage.tsx      # 工作流卡片列表
│           │   ├─ WorkflowDesignerPage.tsx   # 可视化设计器
│           │   ├─ WorkflowRunsPage.tsx       # 执行历史
│           │   ├─ WorkflowRunDetailPage.tsx  # 执行详情
│           │   └─ designer/
│           │       ├─ NodePalette.tsx         # 节点面板
│           │       ├─ NodeConfigPanel.tsx     # 节点配置
│           │       └─ DebugPanel.tsx          # 调试面板
│           ├─ bots/BotsPage.tsx
│           ├─ knowledge/
│           │   ├─ KnowledgeBasesPage.tsx
│           │   ├─ KnowledgeDocumentsPage.tsx
│           │   ├─ KnowledgeDocumentCreatePage.tsx
│           │   └─ KnowledgeDocumentEditPage.tsx
│           ├─ models/ModelProvidersPage.tsx
│           └─ prompts/PromptTemplatesPage.tsx
└─ packages/
    ├─ workflow-schema/                       # @aiworkflow/workflow-schema
    │   └─ 类型定义、校验函数、工厂方法
    ├─ workflow-designer-core/                 # @aiworkflow/workflow-designer-core
    │   └─ 框架无关的设计器逻辑 (布局算法、CRUD 操作)
    ├─ workflow-designer-react/                # @aiworkflow/workflow-designer-react
    │   └─ React 渲染层 (SVG 画布、拖拽、缩放、连线)
    ├─ workflow-designer-vue/                  # @aiworkflow/workflow-designer-vue
    │   └─ Vue 3 适配器 (stub)
    ├─ workflow-designer-wc/                   # @aiworkflow/workflow-designer-wc
    │   └─ Web Component 封装 (<ai-workflow-designer>)
    └─ workflow-sdk/                           # @aiworkflow/workflow-sdk
        └─ 外部 API 客户端 SDK (AiWorkflowClient)

```

workflows设计器包依赖关系:



状态管理策略: - 服务端数据全部通过 `@tanstack/react-query` 管理 (缓存、自动刷新、乐观更新) - 页面级 UI 状态通过 `React useState` 管理 - 无全局状态管理库 (不使用 `Redux/Zustand`)

路由设计: - 不使用 React Router, 采用自定义轻量路由 - `resolveRoute(pathname)` 函数: 字符串路径匹配 → 返回页面组件和标题 - 参数提取: 手动解析 URL (如 `/workflows/:id/designer`) - 导航触发: `history.pushState + dispatchEvent(new PopStateEvent('popstate'))`

数据库设计

平台使用 Flyway 进行版本化的数据库 Schema 管理, PostgreSQL 迁移脚本位于 db/migration/postgresql/ (V1-V31), 达梦位于 db/migration/dameng/ (V1-V5)。

迁移版本历史 (PostgreSQL) :

版本	内容	表数
V1	基础 Schema: workflow, workflow_version, workflow_execution, node_execution, integration_app, webhook_subscription	6
V2	AI 模块: prompt_template, model_provider	2
V3	知识库: vector_store_config, knowledge_base, knowledge_document, knowledge_chunk	4
V4	模型字段扩展: model_type, description, vision_support, price, model	—
V5	模型用途字段: model_usage (CHAT/EMBEDDING)	—
V6	向量表: knowledge_chunk_vector	1
V7	知识库字段: vector_dimension	—
V8	智能体: ai_bot	1
V9	会话消息: bot_session, bot_message	2
V10	ES 连接字段: username, password, api_key, timeouts	—
V11	Bot 可选工作流: workflow_id nullable	—
V12	统一表前缀: 所有表加 agi_ 前缀	—
V14	身份组织认证: tenant、organization、user、role、login_session、auth_audit_log	7+
V17	资产归属与授权: asset_grant	1

变量	默认值	说明
POSTGRES_JDBC_URL	jdbc:postgresql://localhost:5432/aiworkflow	数据库连接
POSTGRES_USERNAME	aiworkflow	数据库用户名
POSTGRES_PASSWORD	aiworkflow	数据库密码
NACOS_DISCOVERY_ENABLED	false	是否启用 Nacos
NACOS_SERVER_ADDR	127.0.0.1:8848	Nacos 地址
FLYWAY_LOCATIONS	classpath:db/migration/postgresql	迁移脚本路径
ORGANIZATION_SERVICE_NAME	organization-service	组织服务名
AUTH_SERVICE_NAME	auth-service	认证服务名

安全与认证架构

平台采用 **Filter 链 + ThreadLocal 上下文** 实现多租户隔离与双轨认证：

请求

→ OpenApiAuthFilter（/api/open/**，校验 X-AGI-App-Code + X-AGI-API-Key）

→ TenantContextFilter（解析租户、用户、角色到 ThreadLocal）

→ Controller

→ TenantBusinessGuard / AssetGrantService（业务资源可见性过滤）

→ 响应后清理 TenantContext / AuthRequestContext

双轨 API：

路径前缀	用途	认证方式
/api/auth/**	登录、刷新、登出、当前用户	用户名密码 → Bearer Token
/api/**（除 open）	管理控制台 CRUD	Bearer Token + 租户上下文
/api/open/**	第三方业务集成	AppCode + ApiKey Header

核心组件：

组件	说明
AuthService	本地登录、Token 刷新、会话管理
TenantContext	ThreadLocal 租户 ID，默认 tenant_default
TenantContextFilter	从 Token 或 Open API 身份解析租户与用户
OpenApiAuthFilter	开放 API 入口认证，写入 OpenApiRequestContext
TenantAdminGuard	平台管理员 / 租户管理员权限校验

组件	说明
AssetGrantService	资产 USE 权限过滤

Spring Security: CSRF 关闭; Filter 层完成鉴权, Security 链默认放行业务 Controller。

生产环境建议在前置网关统一 TLS、限流, 并定期轮换 API Key。

性能调优

后端优化建议: - 数据库连接池: 配置 HikariCP 的 `maximum-pool-size` (默认 10, 高并发建议 20-50) - MyBatis-Plus 分页: 使用 `PaginationInnerInterceptor` 实现物理分页 - 向量搜索: Elasticsearch 的 KNN 搜索性能远优于内存余弦相似度, 生产环境建议使用 ES - 文档处理: 大文件上传使用流式处理, 避免 OOM - 工作流执行: 当前为同步执行, 高并发场景可改为异步 (消息队列 + WebSocket 推送结果)

前端优化建议: - React Query 配置合理的 `staleTime` 和 `gcTime` - 工作流设计器大 DAG 场景使用虚拟化渲染 - 生产构建开启 Vite 的 `code splitting` - Ant Design 组件按需加载 (`tree-shaking`)

第03章 快速开始

环境要求

工具	版本要求	说明
JDK	17+	后端运行环境
Maven	3.8+	后端构建工具
Node.js	18+	前端运行环境
pnpm	8+	前端包管理器
PostgreSQL	15+	生产/标准模式数据库 (可选, local 模式不需要)

本地开发模式 (无数据库)

本地开发模式下, 无需安装任何数据库, 所有数据存储在内存中, 适合快速上手和前/后端联调。

1. 启动后端:

```
cd server
mvn spring-boot:run -Dspring-boot.run.profiles=local
```

等价于:

```
java -jar target/aiworkflow-server-0.1.0-SNAPSHOT.jar --spring.profiles.active=local
```

后端启动后访问：`http://localhost:8080`

验证：`curl http://localhost:8080/v3/api-docs`

2. application-local.yml 核心配置：

```
spring:
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
      - org.springframework.boot.autoconfigure.flyway.FlywayAutoConfiguration
```

通过排除 DataSource 和 Flyway 的自动配置，Spring 容器中不会有 Mapper Bean，StoreConfig 将自动装配所有 InMemory*Store 实例。

标准开发模式（PostgreSQL）

1. 准备 PostgreSQL：

```
# Docker 方式
docker run -d --name pg-aiworkflow \
  -e POSTGRES_DB=aiworkflow \
  -e POSTGRES_USER=aiworkflow \
  -e POSTGRES_PASSWORD=aiworkflow \
  -p 5432:5432 postgres:15
```

2. 启动后端：

```
cd server
mvn spring-boot:run
```

启动时 Flyway 会自动执行 `classpath:db/migration/postgresql/` 下的迁移脚本，创建 16 张表。

3. 达梦数据库模式：

```
cd server
mvn spring-boot:run -Dspring-boot.run.profiles=dameng
```

启动前端

```
cd web

# 首次运行需要安装依赖
pnpm install
```

```
# 启动开发服务器（端口 5173）
pnpm --filter @aiworkflow/admin dev
```

前端启动后访问：`http://localhost:5173`

Vite 自动将 `/api` 和 `/openapi` 请求代理到后端 `http://localhost:8080`。

访问系统

服务	地址
前端管理后台	<code>http://localhost:5173</code>
后端 API	<code>http://localhost:8080</code>
Swagger UI	<code>http://localhost:8080/swagger-ui.html</code>
OpenAPI JSON	<code>http://localhost:8080/v3/api-docs</code>

工程部署

Jar 部署：

```
# 构建
cd server
mvn clean package -DskipTests

# 运行
java -jar target/aiworkflow-server-0.1.0-SNAPSHOT.jar \
  --spring.datasource.url=jdbc:postgresql://your-db-host:5432/aiworkflow \
  --spring.datasource.username=aiworkflow \
  --spring.datasource.password=your-password
```

前端静态部署：

```
cd web
pnpm --filter @aiworkflow/admin build
# 产出在 web/apps/admin/dist/
# 部署到 Nginx/CDN/OBS
```

Nginx 配置示例：

```
server {
  listen 80;
  server_name aiworkflow.example.com;

  # 前端静态文件
  location / {
```

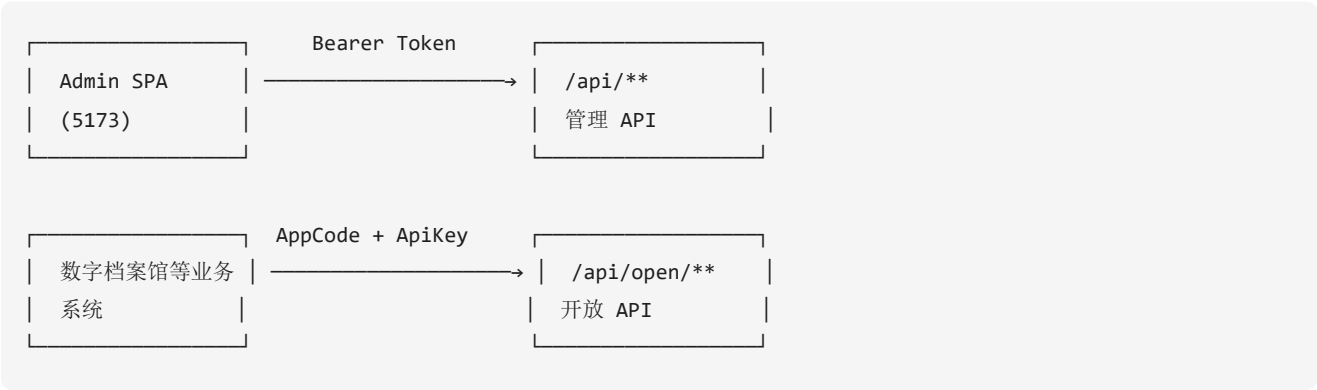
```
root /opt/aiworkflow/dist;
try_files $uri $uri/ /index.html;
}

# 后端 API 代理
location /api/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

第04章 认证与多租户指南

认证体系概览

AI Workflow 同时服务 **管理控制台** 与 **第三方业务系统** 两类调用方：



管理端登录与 Token

登录接口： `POST /api/auth/login`

```
{
  "username": "admin",
  "password": "admin123",
  "tenantCode": ""
}
```

响应字段：

字段	说明
accessToken	访问令牌，请求头 <code>Authorization: Bearer {token}</code>
refreshToken	刷新令牌
expiresIn	访问令牌有效期（秒）

字段	说明
user	当前用户 principal (id、username、tenantId、roleIds)

其他认证接口：

方法	路径	说明
POST	/api/auth/refresh	刷新 Token
POST	/api/auth/logout	注销会话
GET	/api/auth/me	获取当前登录用户

默认租户编码留空时登录 `tenant_default`；首次部署默认管理员账号见种子数据（通常为 `admin / admin123`）。

租户上下文与隔离

`TenantContextFilter` 在每个请求开始时解析身份并写入 `ThreadLocal`：

1. 若命中 Open API 认证 → 使用 `OpenApiRequestContext` 中的 `tenantId / userId / roleIds`
2. 否则解析 Bearer Token → 加载 `LoginSession` 与用户角色
3. 请求结束在 `finally` 块清理上下文，避免线程池污染

业务 Service 通过 `TenantContext.requireTenantId()` 获取当前租户，所有 `agi_*` 业务表均带 `tenant_id` 或在查询时 join 过滤。

组织用户与角色

管理 API 位于 `/api/auth/admin/**` 与 `/api/auth/admin/tenants/{tenantId}/**`：

资源	说明
租户	平台级多租户隔离，仅 <code>platform_admin</code> 可创建
组织	单位/部门树形结构
用户	本地账号，支持启用/禁用/锁定/重置密码
角色	如 <code>platform_admin</code> 、 <code>unit_admin</code> 、 <code>auditor</code>
第三方应用	租户级 Integration App + API Key

开放 API 认证

开放 API 规范见 `docs/openapi/open-api.yaml`。

必需 Header：

Header	说明
X-AGI-App-Code	第三方应用编码
X-AGI-Api-Key	API Key 明文（服务端哈希存储）

可选上下文 Header（用于资产授权过滤）：

Header	说明
X-AGI-User-Id	调用方用户 ID
X-AGI-Unit-Id	单位 ID
X-AGI-Department-Ids	部门 ID 列表
X-AGI-Role-Ids	角色 ID 列表

OpenApiAuthFilter 校验通过后写入运行时身份，后续 Bot/知识库/工作流列表 API 按授权范围返回。

资产授权

表 `agi_asset_grant` 记录资产（BOT / KNOWLEDGE_BASE / WORKFLOW / MODEL_PROVIDER）对组织、部门、角色的 **USE** 权限。

管理入口：GET/POST `/api/asset-grants/**`

业务规则： - 创建者可管理自己创建的资产 - 未配置授权时，同租户内默认可见（可收紧） - 开放 API 携带组织上下文时，仅返回授权范围内资产

审计日志

表 `agi_auth_audit_log` 记录登录、登出、Token 刷新、API Key 使用等事件。

查询入口：GET `/api/system/audit-logs` （默认近 7 天，支持事件类型/用户/结果/时间筛选）

第05章 工作流引擎指南

概念介绍

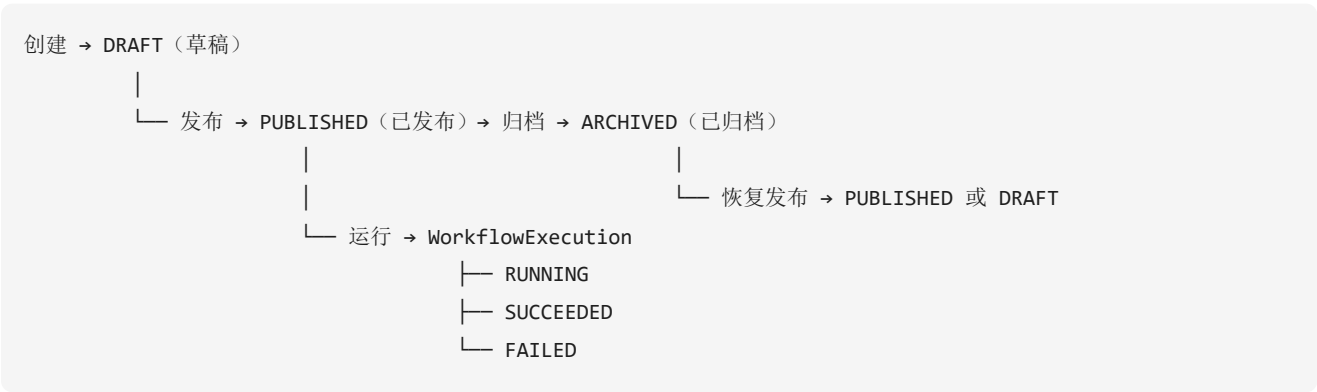
工作流引擎是平台的**核心执行引擎**。用户通过可视化设计器创建由节点（Node）和边（Edge）组成的有向无环图（DAG），每个节点代表一个执行步骤，边定义节点之间的流转关系。

核心概念：

概念	说明
Workflow	工作流实体，包含名称、描述、状态

概念	说明
WorkflowDefinition	工作流定义：节点列表 + 边列表 + 变量列表
WorkflowVersion	不可变的版本快照，每次发布生成新版本
WorkflowNode	工作流节点，有 12 种类型
WorkflowEdge	连接节点的有向边，可带条件表达式
WorkflowVariable	工作流输入/输出变量声明
WorkflowExecution	一次工作流执行记录
NodeExecution	单个节点的执行记录（状态、输入、输出、耗时）

状态流转：



工作流定义模型（DAG）

一个完整的工作流定义由三部分组成：

```
{
  "nodes": [
    {
      "id": "node-start",
      "type": "START",
      "name": "开始",
      "config": {}
    },
    {
      "id": "node-llm-1",
      "type": "LLM",
      "name": "AI 分析",
      "config": {
        "providerId": "provider-xxx",
        "model": "qwen3:8b",
        "systemPrompt": "你是一个数据分析专家",
        "userPrompt": "请分析以下内容: {{ input.text }}",
        "temperature": 0.7,
        "maxTokens": 2000
      }
    }
  ]
}
```

```
    }
  },
  {
    "id": "node-end",
    "type": "END",
    "name": "输出结果",
    "config": {
      "outputKeys": ["analysis"]
    }
  }
],
"edges": [
  {
    "id": "edge-1",
    "sourceNodeId": "node-start",
    "targetNodeId": "node-llm-1"
  },
  {
    "id": "edge-2",
    "sourceNodeId": "node-llm-1",
    "targetNodeId": "node-end"
  }
],
"variables": [
  {
    "name": "text",
    "type": "STRING",
    "required": true
  }
]
}
```

节点类型详解

平台内置 12 种节点类型，每种节点有独立的执行器和配置面板：

1. START（开始节点）

- **执行器**：StartNodeExecutor
- **功能**：工作流入口，将输入参数原样传递到执行上下文
- **数量限制**：每个工作流有且仅有一个
- **配置**：无

2. END（结束节点）

- **执行器**：EndNodeExecutor
- **功能**：工作流出口，从上下文中选择指定的输出字段
- **数量限制**：至少一个

- **配置:** `outputKeys` — 需要输出的字段名列表

3. LLM (大模型调用节点)

- **执行器:** `LlmNodeExecutor`
- **功能:** 调用大语言模型进行推理
- **配置项:**
 - `providerId` — 模型提供商 ID
 - `model` — 模型名称 (如 `qwen3:8b`, `gpt-4`)
 - `systemPrompt` — 系统提示词 (支持 `{{ template }}`)
 - `userPrompt` — 用户提示词 (支持 `{{ template }}`)
 - `temperature` — 温度参数 (0-1)
 - `maxTokens` — 最大输出 Token 数
 - `topP` — 核采样参数
 - `inputParams` — 自定义参数映射

4. PROMPT (提示词模板节点)

- **执行器:** `PromptNodeExecutor`
- **功能:** 对输入文本进行 `{{ variable.path }}` 模板替换
- **配置:** `template` — 模板字符串

5. KNOWLEDGE_RETRIEVAL (知识检索节点)

- **执行器:** `KnowledgeRetrievalNodeExecutor`
- **功能:** 从知识库中检索相关内容
- **配置项:**
 - `knowledgeBaseId` — 知识库 ID
 - `query` — 检索查询 (支持模板语法)
 - `topK` — 返回结果数量

6. HTTP_TOOL (HTTP 工具节点)

- **执行器:** `HttpToolNodeExecutor`
- **功能:** 发送 HTTP 请求到外部 API
- **配置项:**
 - `url` — 请求 URL (支持模板)
 - `method` — 请求方法 (GET/POST/PUT/PATCH)
 - `headers` — 自定义请求头
 - `queryParams` — URL 查询参数
 - `body` — 请求体 (JSON / FORM)
 - `bodyType` — 请求体类型 (JSON / FORM)
 - `timeoutMs` — 超时时间
 - `responseKey` — 响应存储字段名

7. CONDITION (条件判断节点)

- **执行器:** `ConditionNodeExecutor`
- **功能:** 根据条件分支到不同的下游节点
- **配置项:**
 - `conditions` — 条件列表, 每个条件包括:
 - `field` — 判断字段 (模板表达式)
 - `operator` — 运算符: `EQUALS`, `NOT_EQUALS`, `CONTAINS`, `IS_EMPTY`, `IS_NOT_EMPTY`
 - `value` — 比较值
 - `nextNodeId` — 满足条件时的下一个节点

8. TEXT_TRANSFORM (文本转换节点)

- **执行器:** `TextTransformNodeExecutor`
- **功能:** 对文本执行简单的模板替换转换
- **配置:** `template` — 转换模板

9. CONTENT_TEMPLATE (内容模板节点)

- **执行器:** `ContentTemplateNodeExecutor`
- **功能:** 高级模板渲染, 支持 JSON 和 TEXT 两种输出格式
- **配置项:**
 - `template` — 模板内容 (支持循环、条件等复杂语法)
 - `outputFormat` — 输出格式 (JSON / TEXT)

11. QUESTION_CLASSIFIER (问题分类节点)

- **执行器:** `QuestionClassifierNodeExecutor`
- **功能:** 按关键词或模型分类将问题路由到不同分支
- **配置项:**
 - `categories` — 分类项列表 (名称、关键词、描述)
 - `matchMode` — 匹配方式 (关键词 / 模型)
 - `defaultCategoryId` — 未命中时的默认分支
- **输出:** `categoryId`、`categoryName`, 供 `CONDITION` 边或显式 `nextNodeId` 路由

12. LOOP (循环节点)

- **执行器:** `LoopNodeExecutor`
- **功能:** 对数组数据进行循环处理
- **配置项:**
 - `itemsExpression` — 循环数组表达式 (模板语法, 如 `{{ input.items }}`)
 - `subSteps` — 循环体步骤列表 (支持 `CONTENT_TEMPLATE` 和 `HTTP_TOOL`)
 - `maxIterations` — 最大迭代次数

执行引擎原理

`WorkflowExecutionService` 是执行引擎的核心, 执行流程如下:

```

public WorkflowExecutionResult runWorkflow(WorkflowExecutionRequest request) {
    // 1. 加载已发布的版本及其定义
    WorkflowVersion version = workflowService.getPublishedVersion(request.workflowId());
    WorkflowDefinition definition = version.definition();

    // 2. 找到 START 节点，构建节点索引
    WorkflowNode currentNode = findStartNode(definition);
    Map<String, WorkflowNode> nodesById = indexNodes(definition);
    Map<String, List<WorkflowEdge>> outgoingEdges = indexEdges(definition);

    // 3. 创建执行记录 (RUNNING)
    WorkflowExecution execution = createExecution(request, version);
    executionStore.saveWorkflowExecution(execution);

    // 4. DAG 遍历：从 START 走到 END
    Map<String, Object> context = new LinkedHashMap<>(request.input());
    while (currentNode != null) {
        // 4a. 执行当前节点
        NodeExecutionResult nodeResult = executeNode(
            executionId, currentNode, request.input(), context);
        // 4b. 合并节点输出到上下文
        context.putAll(nodeResult.output());

        if (currentNode.type() == WorkflowNodeType.END) break;

        // 4c. 确定下一个节点 (边目标 / 显式 nextNodeId)
        currentNode = nextNode(currentNode, nodeResult, outgoingEdges, nodesById);
    }

    // 5. 标记执行成功
    markSucceeded(execution, finalOutput);
    return buildResult(execution);
}

```

上下文传递模型：

```

输入 input → START → 节点1 输出合并进 Context → 节点2 读取 Context → ... → END 输出
      ↓
    所有节点的 {{ template }} 都从 Context 中取值

```

每个节点执行时会同时收到： - **input**：原始输入参数（不变） - **context**：累计的执行上下文（累积所有前置节点的输出）

DAG 校验机制

`DagValidator` 在工作流创建和发布时进行校验：

校验规则	违规时异常
有且仅有一个 START 节点	DagValidationException
至少有一个 END 节点	DagValidationException
所有节点 ID 唯一	DagValidationException
所有边的 source/target 节点存在	DagValidationException
无循环依赖（DFS 环检测）	DagValidationException

模板渲染引擎

TemplateRenderer 提供统一的模板渲染能力，被 LLM、PROMPT、CONTENT_TEMPLATE、TEXT_TRANSFORM 等节点共用：

语法：{{ variable.path.to.value }}

示例：

输入 Context：

```
{
  "user": { "name": "张三", "age": 30 },
  "items": ["A", "B", "C"]
}
```

模板： "用户 {{ user.name }} 的年龄是 {{ user.age }}"

输出： "用户 张三 的年龄是 30"

核心实现：

```
// 支持嵌套点路径的模板替换
String render(String template, Map<String, Object> context);
// 使用正则匹配 {{ path }} 并通过反射/Map 访问嵌套值
```

版本管理

工作流采用**不可变版本快照**机制：




```
|— definition: JSON 快照 (不可变)
└— status: PUBLISHED
```

- 发布后的版本**不可修改**
- 执行时始终使用**最新发布版本**
- 草稿修改不影响已发布的版本
- 归档后的工作流不可再运行，但可通过 **恢复发布** 回到已发布或草稿状态

恢复发布规则（POST /api/workflows/{id}/restore）：

归档前状态	恢复后状态
曾有 currentVersionId （曾发布）	PUBLISHED
从未发布	DRAFT

工作流 API 参考

方法	路径	说明
GET	/api/workflows	获取工作流列表
POST	/api/workflows	创建工作流
GET	/api/workflows/{id}	获取工作流详情
PUT	/api/workflows/{id}/draft	更新草稿定义
POST	/api/workflows/{id}/publish	发布新版本
POST	/api/workflows/{id}/archive	归档工作流
POST	/api/workflows/{id}/restore	恢复发布（ARCHIVED → PUBLISHED/DRAFT）
POST	/api/workflows/{id}/runs	运行工作流
GET	/api/workflow-runs/{id}	查询执行详情
GET	/api/workflow-runs	执行历史列表

完整 API 文档参考 [docs/openapi/open-api.yaml](#) 与 Swagger UI。

第06章 知识库指南

知识库概念介绍

知识库系统提供了一套完整的文档管理、文本分割、向量嵌入和语义搜索能力，是实现 RAG（检索增强生成）模式的基座。

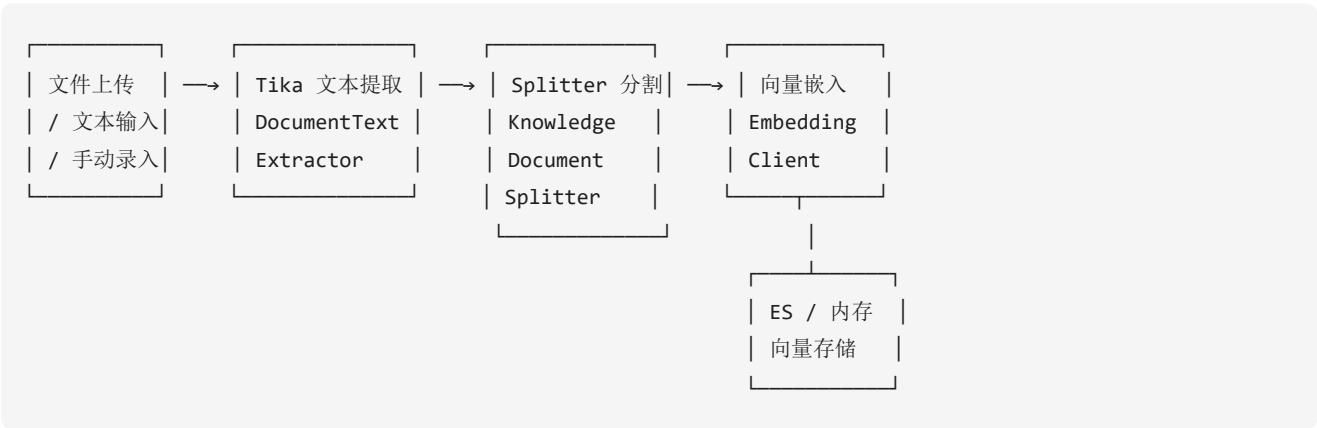
核心概念：

概念	说明
KnowledgeBase	知识库实体，配置嵌入模型、向量存储、分割策略和检索模式
KnowledgeDocument	知识库中的文档（文件上传、手动录入、文本/表格数据集）
KnowledgeChunk	文档分割后的文本片段
KnowledgeChunkVector	分块对应的向量嵌入（JSON 存储）
VectorStoreConfig	向量存储连接配置（Elasticsearch）

数据关系：



文档处理流程



1. 文本提取（DocumentTextExtractor）

基于 Apache Tika，支持的文件格式：

格式	扩展名
纯文本	.txt
Markdown	.md
PDF	.pdf
Word	.docx, .doc
Excel	.xlsx, .xls
CSV/TSV	.csv, .tsv

格式	扩展名
HTML	.html, .htm

2. 文档分割 (KnowledgeDocumentSplitter)

根据配置的分割策略将长文本切分为多个 chunks。

3. 向量嵌入 (EmbeddingClient)

将每个 chunk 的文本转换为向量表示 (float 数组) 。

4. 向量存储 (ElasticsearchVectorStoreClient)

将向量存入 ES 索引或内存中，用于后续的相似度搜索。

文档分割策略

平台提供 5 种分割策略：

策略	枚举值	原理	适用场景
固定长度	FIXED_LENGTH	按字符数等长切割，支持重叠	通用场景，对格式无要求的文本
段落分割	PARAGRAPH	按空行/段落边界分割	结构化文档、文章
语义分割	SEMANTIC	基于语义边界检测分割	需要保持语义完整性的场景
符号分割	SYMBOL	按自定义分隔符（如 ### ）分割	代码文档、有明确标记的文本
结构化表格	STRUCTURED_TABLE	按行分割表格数据	CSV/TSV 表格数据

分割参数：

参数	说明	默认值
chunkSize	每个 chunk 的最大字符数	200 (文本) / 0 (通用)
chunkOverlap	相邻 chunk 的重叠字符数	-1 (自动)
separator	符号分割时的分隔符	无

向量嵌入 (Embedding)

嵌入客户端接口：

```
public interface EmbeddingClient {
    float[] embed(String text); // 单文本嵌入
    List<float[]> embedAll(List<String> texts); // 批量嵌入
}
```

两种实现：

实现类	说明	维度	适用场景
LocalEmbeddingClient	基于字符哈希的本地嵌入	64 维	开发/测试/无外部依赖
ModelProviderEmbeddingClient	通过 Ollama 等模型提供商 API	取决于模型	生产环境

ModelProviderEmbeddingClient 工作流程：

```
Chunk → HTTP POST → Ollama /api/embeddings → 返回向量 → 存入 VectorStore
↓
请求体: { "model": "nomic-embed-text", "prompt": "chunk文本" }
响应: { "embedding": [0.12, -0.34, ...] }
```

向量存储与搜索

向量存储客户端 (ElasticsearchVectorStoreClient)：

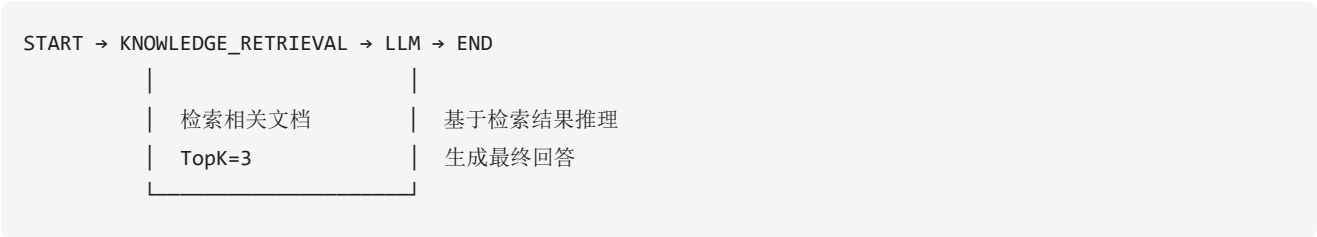
```
// 核心操作
void upsert(String indexName, String chunkId, float[] vector, Map<String, Object> metadata);
void delete(String indexName, String chunkId);
List<KnowledgeSearchResult> search(String indexName, float[] queryVector, int topK);
```

三种检索模式：

模式	实现方式	适用场景
关键词检索 (keyword)	文本匹配	精确匹配、已知术语搜索
向量检索 (vector)	余弦相似度 / ES KNN	语义搜索、模糊匹配
混合检索 (hybrid)	关键词 + 向量融合	综合最优效果

知识库与工作流联动

在 Workflow 中使用 KNOWLEDGE_RETRIEVAL 节点即可在工作流中调用知识库：



典型 RAG Pipeline： 1. 用户输入问题 2. KNOWLEDGE_RETRIEVAL 节点从知识库检索 Top K 相关文档片段 3. LLM 节点将检索结果作为 Context 拼接进 Prompt 4. LLM 基于 Context 生成答案

知识库 API 参考

方法	路径	说明
GET	/api/knowledge-bases	获取知识库列表
POST	/api/knowledge-bases	创建知识库
PUT	/api/knowledge-bases/{id}	更新知识库
DELETE	/api/knowledge-bases/{id}	删除知识库
GET	/api/knowledge-bases/{id}/documents	获取文档列表
POST	/api/knowledge-bases/{id}/documents	添加文档（文本）
POST	/api/knowledge-bases/{id}/documents/upload	上传文档（文件）
POST	/api/knowledge-bases/{id}/documents/text/upload	上传纯文本数据集
POST	/api/knowledge-bases/{id}/documents/table/upload	上传表格数据集
POST	/api/knowledge-bases/{id}/documents/manual	手动录入数据集
DELETE	/api/knowledge-bases/{id}/documents/{docId}	删除文档
POST	/api/knowledge-bases/{id}/documents/{docId}/reparse	重新解析文档
GET	/api/knowledge-bases/{id}/documents/{docId}/chunks	查询分块列表
PUT	/api/knowledge-bases/{id}/chunks/{chunkId}	更新分块
POST	/api/knowledge-bases/{id}/search	搜索知识库
POST	/api/knowledge-bases/{id}/embeddings/backfill	回填向量嵌入
POST	/api/knowledge-bases/chunks/preview	分块效果预览
POST	/api/knowledge-bases/documents/upload/preview	文档上传预览
POST	/api/knowledge-bases/documents/text/upload/preview	文本数据集预览
POST	/api/knowledge-bases/documents/table/upload/preview	表格数据集预览

完整 API 文档参考 [docs/openapi/open-api.yaml](#) 与 Swagger UI。

第07章 智能体（Bot）指南

智能体架构

智能体（Bot）是面向最终用户的 AI 对话机器人，可以连接 workflow、知识库和模型提供商，实现多种交互模式。

AiBot 数据结构：

AiBot		
└─ name, description, avatar	#	基本信息
└─ status: ENABLED DISABLED	#	启用/禁用
└─ workflowId → Workflow	#	关联工作流（可选）
└─ modelProviderId → ModelProvider	#	关联模型（可选）
└─ knowledgeBaseId → KnowledgeBase	#	关联知识库（可选）
└─ systemPrompt	#	系统提示词
└─ openingMessage	#	开场白

三种协作模式：

模式 1：工作流驱动		
Bot	→ 工作流 → 知识检索 + LLM 推理 + HTTP 工具 →	返回结果
模式 2：直接 LLM 对话		
Bot	→ 模型提供商 → 返回 AI 回复	
模式 3：RAG 增强对话		
Bot	→ 知识库检索 → 拼接 Context → LLM 推理 →	返回结果

会话管理

每个 Bot 可以创建多个会话（Session），每个会话包含一组有序的消息。

BotSession：

字段	类型	说明
id	string	会话 ID
botId	string	所属 Bot ID
title	string	会话标题
messageCount	int	消息数量
createdAt	datetime	创建时间
updatedAt	datetime	最后活跃时间

BotMessage：

字段	类型	说明
id	string	消息 ID
sessionId	string	所属会话 ID
botId	string	所属 Bot ID
role	USER ASSISTANT	消息角色

字段	类型	说明
content	string	消息内容
createdAt	datetime	发送时间

执行模式详解

模式 1： 工作流模式 (POST /api/bots/{id}/run)

用户发送消息

→ BotService.run()

→ 加载 Bot 关联的 Workflow

→ 将用户消息封装为工作流输入

→ WorkflowExecutionService.runWorkflow()

→ 返回执行结果 + 工作流输出

模式 2： 对话模式 (POST /api/bots/{id}/chat)

用户发送消息

→ BotService.chat()

→ 查找或创建会话

→ 加载历史消息

→ 两种情况：

└ 有工作流：通过工作流执行生成回复

└ 无工作流：直接调用 ChatModelClient 生成回复

→ 保存用户消息 + AI 回复

→ 返回 {session, messages, reply, execution}

Chat 对话流程

```
public BotChatResult chat(String botId, String sessionId, String message, Map<String, Object> input) {
    // 1. 加载 Bot
    AiBot bot = botStore.findById(botId);

    // 2. 获取或创建会话
    BotSession session = findOrCreateSession(botId, sessionId);

    // 3. 保存用户消息
    saveMessage(session.id(), botId, USER, message);

    // 4. 生成回复
    BotMessage reply;
    WorkflowExecutionResult execution = null;

    if (bot.workflowId() != null) {
        // 工作流模式
        execution = executionService.runWorkflow(new WorkflowExecutionRequest(
            bot.workflowId(), buildWorkflowInput(bot, message, input)));
    }
}
```

```
        reply = buildReplyFromExecution(execution, bot);
    } else {
        // 直接 LLM 模式
        String replyText = chatModelClient.generate(
            bot.modelProviderId(), null,
            buildChatPrompt(bot, session, message));
        reply = new BotMessage(UUID.randomUUID().toString(), session.id(), botId,
            ASSISTANT, replyText, Instant.now());
    }

    // 5. 保存回复
    saveMessage(reply);

    // 6. 返回完整结果
    return new BotChatResult(session, listMessages(session.id()), reply, execution);
}
```

智能体 API 参考

方法	路径	说明
GET	/api/bots	获取智能体列表
POST	/api/bots	创建智能体
PUT	/api/bots/{id}	更新智能体
DELETE	/api/bots/{id}	删除智能体
POST	/api/bots/{id}/run	运行智能体（工作流模式）
POST	/api/bots/{id}/chat	与智能体对话
GET	/api/bots/{id}/sessions	获取会话列表
GET	/api/bots/{id}/sessions/{sid}/messages	获取会话消息

完整 API 文档参考 [docs/openapi/open-api.yaml](#) 与 Swagger UI。

第08章 模型提供商指南

模型体系介绍

模型提供商模块统一管理所有 LLM 和 Embedding 模型的连接配置。平台支持两个维度的模型：

模型用途 (modelUsage)：

用途	说明	对应接口
CHAT	对话/推理模型，用于 LLM 节点和 Bot 对话	ChatModelClient

用途	说明	对应接口
EMBEDDING	向量嵌入模型，用于知识库的文本向量化	EmbeddingClient

模型类型 (modelType): - CUSTOM：通用的 OpenAI 兼容 API - OLLAMA：Ollama 私有部署模型

模型类型与用途

ModelProvider 数据结构：

字段	类型	必填	说明
name	string	是	提供商名称
modelType	string	是	模型类型 (CUSTOM/OLLAMA)
modelUsage	string	否	用途 (CHAT/EMBEDDING)
description	string	否	描述
visionSupport	boolean	否	是否支持视觉输入
pricePerMillionTokens	number	否	每百万 Token 价格
baseUrl	string	是	API 地址 (如 http://localhost:11434)
model	string	是	模型名 (如 qwen3:8b)
apiKeyRef	string	是	API Key 引用
enabled	boolean	否	是否启用

接入 Ollama 私有部署

1. 安装 Ollama 并拉取模型：

```
# 安装 Ollama (Windows/Mac/Linux: https://ollama.com)
ollama pull qwen3:8b           # 对话模型
ollama pull nomic-embed-text   # 嵌入模型
```

2. 在平台中配置模型提供商：

对话模型：

```
POST /api/model-providers
{
  "name": "Ollama-Qwen",
  "modelType": "OLLAMA",
  "modelUsage": "CHAT",
  "baseUrl": "http://localhost:11434",
  "model": "qwen3:8b",
  "apiKeyRef": "ollama",
}
```

```
"enabled": true
}
```

嵌入模型：

```
POST /api/model-providers
{
  "name": "Ollama-Embed",
  "modelType": "OLLAMA",
  "modelUsage": "EMBEDDING",
  "baseUrl": "http://localhost:11434",
  "model": "nomic-embed-text",
  "apiKeyRef": "ollama",
  "enabled": true
}
```

3. 在工作流/知识库中使用：

- 创建知识库时，选择 `Ollama-Embed` 作为嵌入模型
- 在工作流的 LLM 节点中，选择 `Ollama-Qwen` 作为对话模型

模型提供商 API 参考

方法	路径	说明
GET	<code>/api/model-providers</code>	获取模型提供商列表
POST	<code>/api/model-providers</code>	创建模型提供商
PUT	<code>/api/model-providers/{id}</code>	更新模型提供商
DELETE	<code>/api/model-providers/{id}</code>	删除模型提供商

第09章 智能编研与开放集成

编研模板与 workflow 绑定

编研模块（`generation` 包）提供专题成果结构化定义：

表	说明
<code>agi_generation_template</code>	模板 schema、章节 instruction、绑定 workflow ID
<code>agi_generation_job</code>	编研任务实例
<code>agi_generation_output</code>	生成成果（Markdown / DOCX）

模板保存时会快照绑定 workflow 的节点结构，避免后续 workflow 变更影响已发布模板。

管理 API: /api/generation-templates
编研向导 API: /api/research/**

智能编研任务链路

智能编研向导（5 步）

→ ResearchController 提交任务

→ 创建 GenerationJob

→ 调用绑定工作流 WorkflowExecutionService.runWorkflow()

→ 知识库检索 / HTTP / LLM / 内容模板 等节点

→ 汇总章节大纲与正文

→ 写入 GenerationOutput（支持 DOCX 导出）

第三方应用与 Java SDK

Maven 依赖（client/aiworkflow-open-api-client）：

```
<dependency>
  <groupId>com.mw.ai.agi</groupId>
  <artifactId>aiworkflow-open-api-client</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</dependency>
```

配置（application.yml）：

```
agi:
  openapi:
    enabled: true
    base-url: http://localhost:8080
    app-code: your-app-code
    api-key: your-api-key
```

自动注册 Feign Client: AgiOpenBotClient、AgiOpenKnowledgeBaseClient、AgiOpenWorkflowClient 等。

TypeScript 侧可使用 @aiworkflow/workflow-sdk 的 AiWorkflowClient。

设计器嵌入 (React/Vue/WC)

包	场景
@aiworkflow/workflow-designer-react	React 应用内嵌完整设计器
@aiworkflow/workflow-designer-vue	Vue 3 嵌入
@aiworkflow/workflow-designer-wc	任意 HTML 页面 Web Component
@aiworkflow/workflow-schema	共享类型与 DAG 校验

详见 [docs/integration-guide.md](#)。

第10章 二次开发指南

Store 模式扩展

当需要为新的业务模块添加持久化切换能力，需要以下步骤：

1. 定义 Store 接口：

```
// com.mw.ai.agi.yourmodule.service.YourStore.java
public interface YourStore {
    YourDomain save(YourDomain domain);
    Optional<YourDomain> findById(String id);
    List<YourDomain> list();
    void delete(String id);
}
```

2. 实现 InMemory 版本：

```
// com.mw.ai.agi.yourmodule.service.InMemoryYourStore.java
public class InMemoryYourStore implements YourStore {
    private final ConcurrentHashMap<String, YourDomain> store = new ConcurrentHashMap<>();

    @Override
    public YourDomain save(YourDomain domain) { store.put(domain.id(), domain); return domain; }

    @Override
    public Optional<YourDomain> findById(String id) { return Optional.ofNullable(store.get(id)); }

    @Override
    public List<YourDomain> list() { return List.copyOf(store.values()); }

    @Override
    public void delete(String id) { store.remove(id); }
}
```

3. 实现 MyBatis 版本：

```
// com.mw.ai.agi.yourmodule.persistence.YourEntity.java
@TableName("agi_your_table")
public class YourEntity {
    @TableId private String id;
    // ... fields with @TableField matching DB columns
}

// com.mw.ai.agi.yourmodule.persistence.YourMapper.java
```

```

@Mapper
public interface YourMapper extends BaseMapper<YourEntity> {}

// com.mw.ai.agi.yourmodule.service.MybatisYourStore.java
public class MybatisYourStore implements YourStore {
    private final YourMapper mapper;
    // implement CRUD using mapper
}

```

4. 在 StoreConfig 中注册：

```

@Bean
public YourStore yourStore(ObjectProvider<YourMapper> mapperProvider) {
    YourMapper mapper = mapperProvider.getIfAvailable();
    return mapper == null ? new InMemoryYourStore() : new MybatisYourStore(mapper);
}

```

新增自定义节点类型

平台支持通过实现 `WorkflowNodeExecutor` 接口来添加新的节点类型。

步骤：

1. 在 WorkflowNodeType 枚举中添加新类型：

```

// server/.../workflow/domain/WorkflowNodeType.java
public enum WorkflowNodeType {
    // ... existing types
    EMAIL_SENDER    // 新增：邮件发送节点
}

```

2. 实现执行器：

```

// com.mw.ai.agi.workflow.engine.EmailSenderNodeExecutor.java
@Component
public class EmailSenderNodeExecutor implements WorkflowNodeExecutor {

    @Override
    public WorkflowNodeType nodeType() {
        return WorkflowNodeType.EMAIL_SENDER;
    }

    @Override
    public NodeExecutionResult execute(WorkflowNode node, NodeExecutionContext context) {
        // 1. 从 node.config() 中读取配置
        Map<String, Object> config = node.config();
        String to = TemplateRenderer.render((String) config.get("to"), context.context());
        String subject = TemplateRenderer.render((String) config.get("subject"), context.context());
        String body = TemplateRenderer.render((String) config.get("body"), context.context());
    }
}

```

```

        // 2. 执行业务逻辑
        sendEmail(to, subject, body);

        // 3. 返回结果
        return new NodeExecutionResult(Map.of("emailSent", true, "recipient", to));
    }
}

```

3. 注册即生效： `WorkflowNodeExecutorRegistry` 通过构造函数注入 `List<WorkflowNodeExecutor>`，Spring 会自动发现所有 `@Component` 标注的实现类，无需额外配置。

4. 前端适配：

- 在 `workflow-schema` 包的类型定义中添加新节点类型
- 在 `NodePalette.tsx` 的节点列表中注册新节点
- 在 `NodeConfigPanel.tsx` 中添加新节点的配置表单

自定义 Embedding 客户端

如需接入新的嵌入服务（如 OpenAI Embedding、HuggingFace 等）：

```

// com.mw.ai.agi.knowledge.service.OpenAiEmbeddingClient.java
@Component
public class OpenAiEmbeddingClient implements EmbeddingClient {

    private final RestTemplate restTemplate = new RestTemplate();

    @Override
    public float[] embed(String text) {
        return embedAll(List.of(text)).get(0);
    }

    @Override
    public List<float[]> embedAll(List<String> texts) {
        // 调用 OpenAI Embedding API
        var request = Map.of(
            "model", "text-embedding-ada-002",
            "input", texts
        );
        var response = restTemplate.postForObject(
            "https://api.openai.com/v1/embeddings", request, Map.class);
        // 解析返回的向量
        return parseVectors(response);
    }
}

```

自定义文档分割器

要实现新的分割策略，需要在 `KnowledgeDocumentSplitter` 中添加新的分割逻辑：

```
// 在 KnowledgeDocumentSplitter 中添加新的分割方法
private List<KnowledgeChunk> splitByCustomStrategy(String text, KnowledgeSplitRequest request) {
    // 自定义分割逻辑
    // 返回 List<KnowledgeChunk>
}
```

并在 `KnowledgeBaseService.create()` 和 `addDocument()` 方法中注册新的 `splitterType`。

Feign 外部服务集成

1. 定义 Feign 客户端：

```
// com.mw.ai.agi.integration.weather.WeatherClient.java
@FeignClient(name = "${weather.service-name:weather-service}",
            url = "${weather.base-url:}")
public interface WeatherClient {

    @GetMapping("/api/weather")
    WeatherResponse getWeather(@RequestParam("city") String city);

    record WeatherResponse(String city, double temperature, String condition) {}
}
```

2. 在 application.yml 中配置：

```
weather:
  service-name: weather-service
  base-url: http://weather-api.example.com
```

3. 创建服务包装类并注入到节点执行器：

```
@Service
public class WeatherService {
    private final WeatherClient weatherClient;

    public WeatherService(WeatherClient weatherClient) {
        this.weatherClient = weatherClient;
    }

    public WeatherClient.WeatherResponse getWeather(String city) {
        return weatherClient.getWeather(city);
    }
}
```

第11章 API 接口参考

通用约定

Base URL: http://localhost:8080

统一响应格式:

```
// 成功
{
  "success": true,
  "data": { ... },
  "error": null
}

// 失败
{
  "success": false,
  "data": null,
  "error": {
    "code": "WORKFLOW_NOT_FOUND",
    "message": "Workflow not found: xxx",
    "requestId": "req-123",
    "details": {}
  }
}
```

分页格式:

```
{
  "success": true,
  "data": {
    "items": [ ... ],
    "total": 100
  },
  "error": null
}
```

HTTP 状态码:

状态码	含义
200	请求成功
400	请求参数错误 / 校验失败 / DAG 非法
404	资源未找到
500	服务器内部错误

接口总览

共 20+ 个 Controller，管理 API 与开放 API 分组如下：

分组	Controller 示例	基础路径
工作流	WorkflowController, WorkflowRunController	/api/workflows , /api/workflow-runs
知识库	KnowledgeBaseController, VectorStoreConfigController	/api/knowledge-bases
智能体	BotController	/api/bots
模型/Prompt	ModelProviderController, PromptTemplateController	/api/model-providers , /api/prompts
编研	GenerationTemplateController, ResearchController	/api/generation-templates , /api/research
认证管理	AuthController, AuthAdminController	/api/auth , /api/auth/admin
资产/系统	AssetGrantController, SystemMenuController, DataDictionaryController	/api/asset-grants , /api/system/**
开放 API	OpenBotController, OpenWorkflowController, OpenKnowledgeBaseController	/api/open/**

开放 API 规范

完整 OpenAPI 3.0 定义： docs/openapi/open-api.yaml

在线文档： - Swagger UI： http://localhost:8080/swagger-ui.html - 管理 API 分组： management - 开放 API 分组： open-api

用户操作说明见 [aiworkflow-user-manual.md](#)

通用错误码

错误码	说明
VALIDATION_ERROR	请求参数校验失败
WORKFLOW_NOT_FOUND	工作流未找到
WORKFLOW_RUN_NOT_FOUND	工作流执行记录未找到
INVALID_WORKFLOW_DAG	工作流 DAG 校验失败
INTERNAL_ERROR	服务器内部错误

相关文档： - [aiworkflow-user-manual.md](#) — 用户操作手册 - [integration-guide.md](#) — 集成与本地启动 - [openapi/open-api.yaml](#) — 开放 API 规范